

Team Name



Technology Stack: Python | Big Data

Supervising Faculty: Sir Syed Zaman

Team Member

S No.	Student ID	Full Name	Contact Email / Mobile
01	STD-1700999	Roman Ali	+923043491759/ romanvistro110@gmail.com
02	STD-1702814	Muhammad Hamza Mughal	+923330246861/ mughalhamza065@gmail.com
03	STD-1696663	Bisma Batool	+923182323456/ bismahbatool11@gmail.com
04	STD-1702150	Warisha Rehman	+923142591929/ wareeshaarehman@gmail.com
05	STD-169775	Abdul Moeed Ahmed	+923153550155/ moeedahmed222@gmail.com

TABLE OF CONTENT

1. Introduction	6
1.1 Problem Definition	6
1.2 Background & Necessity	6
1.3 Proposed Solution	7
1.4 Purpose of the Document	7
1.5 Scope	7
1.6 Assumptions	7
1.7 Constraints	7
2. Requirements	8
2.1 Functional Requirements	8
2.2 Non-Functional Requirements	8
3. System Architecture	9
3.1 Application Architecture	10
3.2 Big Data Architecture	10
3.3 Data Flow	11
4. Database Design	12
4.1 Data Dictionary/ Entity Relationship	13
4.3 Dataset Profile	14
5. Behavioural Models	15
5.1 Use-Case Diagram	15
5.2 Activity Diagram	16
5.3 Sequence Diagram	17
6. Methodology	17
6.1 Dataset Generation	17
6.2 Data Quality & Cleaning	19
6.3 Spark Processing & Features	20
6.4 Route Performance Scoring	21
6.5 Bottleneck Detection	24
6.6 Analytics	25
6.7 Spark SQL	26
6.8 Passenger Flow	28
6.9 Origin-Destination Analysis	28
6.10 Delay Analysis	30
6.11 Anomaly Detection	32
6.12 Travel Time, Scheduled Against Actual	35
6.13 Headway, Bunching and Service	36
6.14 Service Frequency and the Demand-	37
7. Machine-Learning Models	38
7.1 How the Data Is Split	38
7.2 Dual Pipeline , Overcrowding Prediction	39
7.3 Delay Severity Prediction	41
7.4 Demand Forecasting	43

7.5 Route Clustering	44
7.6 Model Version Tracking	45
7.7 Persistent Overcrowding	45
7.8 Crowding Risk	45
8. Recommendation Engine	46
8.1 What-If Scenario Impact	48
9. Testing, Security & Privacy	49
9.1 Testing	50
9.2 Security	50
9.3 Privacy	50
9.4 Processing Log and Timing Evidence	50
10. Limitations & Future Enhancements	52
10.1 Limitations	52
10.2 Future Enhancements	52
11. Conclusion	53

Project Declaration:

Herein, we present the project entitled “UrbanTransit IQ Big Data and Data Science Analytics for Public Transport,” which has been created as a platform for Big Data and Data Science analytics through the use of Python, Apache Spark (PySpark, Spark SQL, Spark MLlib), SQL Server, MongoDB, and Power BI. The objective of this project is to perform analysis on large public transport data in terms of tickets, routes, schedules, number of passengers, and delays.

The primary goal of the project is to build a data analysis system for transport administrators and operators that would allow them to find out the peaks of traffic, routes where there are too many or too few passengers, delay patterns, etc., and then turn all this information into recommendations for scheduling and capacities management.

UrbanTransit IQ analyzes one-year worth of data generated for the network simulation with the help of twelve interdependent tables, containing two million ticketing transactions and more than half a million trips. There are two independent analysis pipelines used in UrbanTransit IQ based on Apache Spark and Spark MLlib and Python, scikit-learn and XGBoost frameworks.

In addition, the project features a role-based web dashboard using React , wherein operators, analysts, evaluators, and administrators get to monitor route performance, delay patterns, forecasting results, system-generated suggestions, and a simulator to test operational scenarios.

In this regard, we hereby certify that this project is an original work produced solely for the purpose of the TechWiz-7 competition in the Data Science Intelligence Arena category, and all data utilized herein have been artificially created for this project.

Acknowledgement:

It is our pleasure to thank the supervision of our faculty, Sir Syed Zaman for constant guidance and support throughout the making of our application "UrbanTransit IQ." We are also grateful to Aptech Shahrah e Faisal Center for providing us with all the necessary means of this project.

It is with great pleasure that we would like to mention the combined effort of our team, namely: Hamza Mughal, Roman, Warisha, Bisma Batool, and Abdul Moeed, on working together in generating dataset, using Spark, analytics, machine learning, and the dashboard, which resulted in the making of UrbanTransit IQ.

The creation of UrbanTransit IQ allowed our team to investigate and utilize a vast array of technical concepts. Throughout this project, we worked with such tools as Apache Spark, Spark SQL, and Spark MLlib

for distributed data processing, as well as the separate pipeline consisting of Python, scikit-learn, and XGBoost for benchmarking. We also investigated such topics as data quality engineering, feature engineering at different analytical granularities, dual pipeline model validation, demand prediction, and role-based access control. We appreciate very much the opportunity to be able to do research in an area that combines Big Data engineering and Data Science modeling. This helped us improve not only our technical abilities but also our capacity to question and critically evaluate the limits of our approach.

Executive Summary:

The amount of data generated by public transportation systems is immense, from card taps to GPS signals to delay records and timings, but the data itself does not often translate to any action or decision made. The creation of UrbanTransit IQ aimed at addressing this issue through a platform based on Big Data and Data Science, where all of the simulation results for one year of public transportation operations are used to make decisions based on data

The architecture revolves around two separate processing pipelines dealing with the same problems independently, one based on Apache Spark / PySpark / Spark MLlib, and the other on Python with pandas, scikit-learn and XGBoost, allowing one to verify whether the results of one pipeline match the results of another rather than relying blindly on either. In terms of core problem of prediction of overcrowding events, both pipelines agree on most of unseen trips and achieve combined accuracy of 87%, comfortably exceeding the threshold set by the project.

Apart from predicting events, the system allows to forecast passenger demand and vehicle occupancy levels, cluster routes, detect anomalies, simulate what-if scenarios, and use a role-based dashboard for operators, analysts, evaluators and admins. Every step of the pipeline from dataset generation to data cleaning, feature extraction, modelling and recommendation is logged, tested, and reproducible from a fixed random seed.

It should be noted that the research is also quite clear about where the shortcomings of its results compared to the target values lie, instead of trying to manipulate the data in such a way as to cover up these shortcomings. Specifically, the delay-severity model, as illustrated by the results obtained from the simulation of the system, turns out to be very close to the mathematical limit of predictability possible with such data.

1. Introduction

1.1 Problem Definition

Public transport operators generate large volumes of ticketing, route, timing, passenger-count and delay data, and periodic reports rarely reveal the relationships between demand, occupancy, delays and schedule efficiency. UrbanTransit IQ processes a full year of network data to surface peak periods, overcrowded and underutilized routes, delay patterns, passenger flows and concrete operational actions.

The question the platform is built around is not which route carried the most passengers last week. That is a single aggregation and a spreadsheet answers it. The harder version has four parts: which routes run half-empty at the wrong times, so capacity is paid for and not used; which are packed on weekdays and empty at weekends, so the average hides both problems; where delays originate rather than where they are observed; and whether any of that can be separated from ordinary variation, because a year of a network this size produces enough noise to support almost any story told selectively.

1.2 Background & Necessity

Classic analyses of public transportation systems rely heavily on report cards and manual creation of spreadsheets. Such methods have trouble detecting the interdependencies between passenger demand, travel time, occupancy, delays, and usage by stops once a network has thousands of trips daily.

UrbanTransit IQ analyzes an entire year of simulations of public transportation systems in order to detect peak travel times, overloaded and underused routes, delays, passenger flows, bottlenecks, and other features that can be used for schedule creation.

1.3 Proposed Solution

UrbanTransit IQ is a Big Data and Data Science-driven analytics engine, consisting of two separate pipelines of data processing, both of which independently resolve the same task, therefore, enabling validation of their results rather than blind trust in them:

Big Data pipeline: Apache Spark, PySpark and Spark SQL (optionally HDFS), for ingestion, cleansing, joining and feature engineering of millions of rows, and for one ML pipeline altogether (Spark MLlib).

Python Data Science pipeline: pandas, scikit-learn and XGBoost, which are trained independently of each other on the same tables underneath, never reusing Spark's forecasts.

Responses from the two pipelines are compared automatically.

1.4 Purpose of the Document

This report contains details about what UrbanTransit IQ measures, how it is constructed, what measurements it performs, and where its measurements become invalid. The report is constructed such that any figure presented herein can be mapped back to the artifact it came from: all figures are parsed directly from the output of the pipeline rather than manually inserted, hence re-running the pipeline and recreating the report cannot leave an out-of-date figure in the report.

1.5 Scope

Scope: Data set creation, Spark processing, data quality and cleansing, feature creation, analytics, forecasting, clustering, dual pipeline modeling, recommendations, visualization and RBAC. Out of scope: Live Ticketing, real time GPS, Payment System Integration.

1.6 Assumptions

- The results will be based on the level of real-world data contained in the generated dataset
- Separate training processes for Spark and Python algorithms are not anticipated to be identical.
- Delay and demand will be impacted by inherently noisy conditions (weather, traffic, etc.)
- The passenger data, ticketing data, and location data are all processed according to the data protection guidelines.

1.7 Constraints

Project effectiveness relies on the quality, accuracy, and reality of the produced ticketing, routes, schedules, passengers' numbers, and delays data, as well as the computing resources used for the Spark operations.

Differences might happen between the output data of the Spark/PySpark pipeline and the output data of the separate Python pipeline, that is normal and is measured and reported, unlike hidden.

The prediction of the demand and delay is restricted by how much randomness (accidents, weather conditions, special occasions) is introduced into the process by the simulation, in the subsequent section, there is more information about what point the delay prediction might reach without exceeding the limit.

As the passenger and ticketing data are simulated, rather than collected in real life, privacy and security issues are only demonstrated on the design level.

2. Requirements

2.1 Functional Requirements

- Authentication and role-based access (administrator/analyst/operator).
- Ingestion, validation, quality check and cleaning of Big Data.
- Spark SQL, partitioning and Parquet/Hadoop Distributed File System (HDFS).
- Feature engineering.
- Overcrowding, route scoring and categorization.
- Demand prediction, occupancy, crowding risk and clustering of routes.
- Double pipeline modeling and comparisons with recommendations.
- Scenario what-if analysis.
- Dashboards, search, reports and export.

2.2 Non-Functional Requirements

Performance: the system deals with millions of ticketing and passenger movement records. The dashboards will be optimized such that they will provide results in a few seconds after processing data.

Scalability: the system is developed in such a way that it can grow to process much more records without any redesign as the process carried out by Spark is independent of the dataset's size.

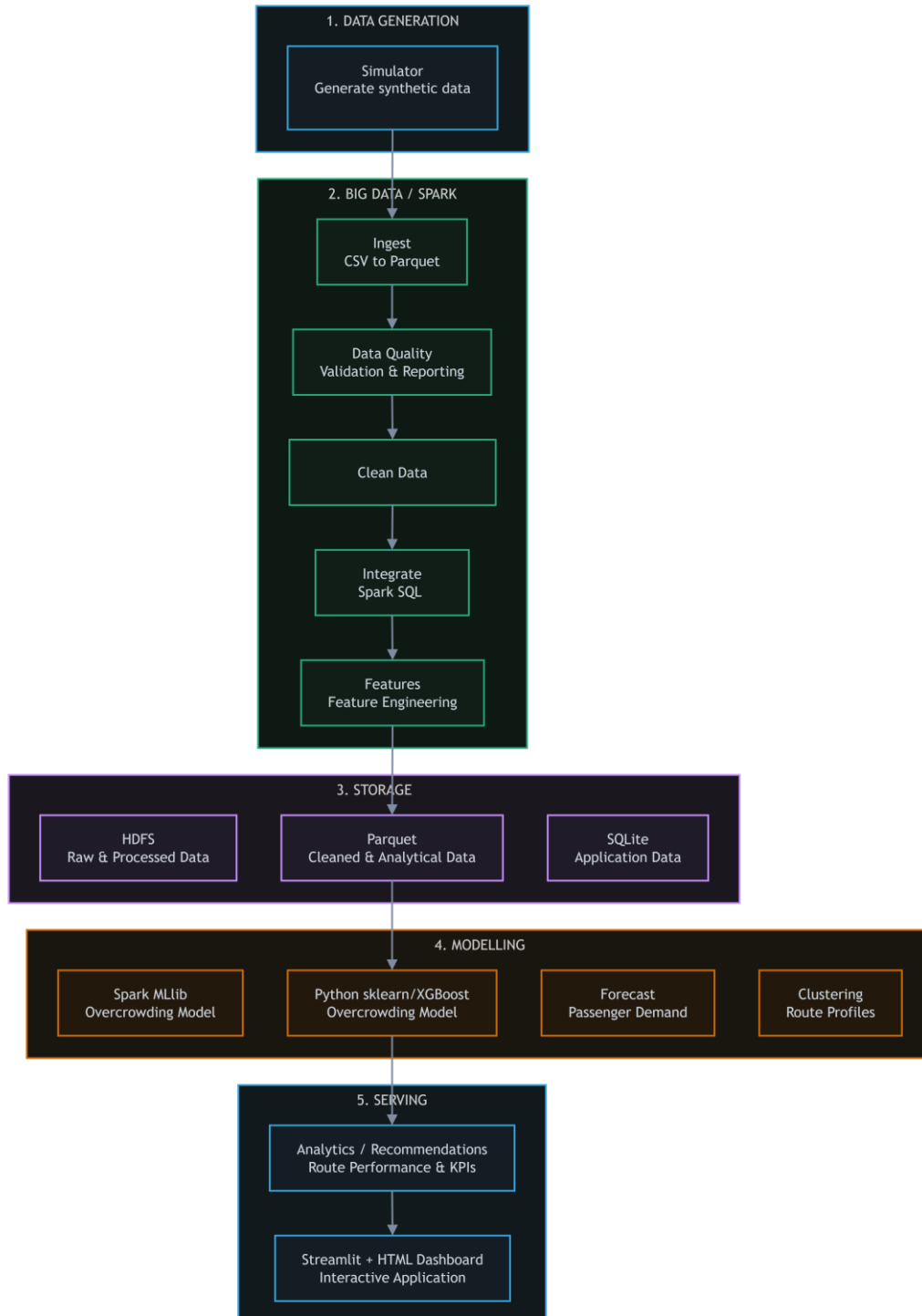
Usability: role-based React dashboard which provides operators, analysts, evaluators, and administrators a dashboard appropriate for their roles.

Accuracy: classification and prediction models will be tested on held-out chronologically separated test dataset.

Availability: application will be running locally/deployed React app designed to stay available during the evaluation.

3. System Architecture

This architecture consists of a pipeline architecture comprising data creation, Big Data analysis using Spark, data storage in Parquet/HDFS, modeling, and serving.



3.1 Application Architecture

Module	Responsibility
settings.py	all threshold and generators' parameters in one place
schemas.py	Schema definition, primary and foreign keys, partitioning
paths.py	raw, preprocessed, Parquet, model and reports artifacts location
spark.py	session creation including the workaround for Windows/HDFS
registry.py	versioned models storing with metrics and training environment
store.py	SQLite: run logs, metrics, audit trail
accounts.py	Roles, abilities, and password hash
simulate/	Simulation of dataset: networks, operations, defects, and ceiling measurement
pipeline/	Ingestion, data quality, cleaning, integration, features, and catalog
analysis/	Route performance, network behavior, and clustering
models/	Double overcrowding, delay magnitude, prediction, and agreement
advice/	Recommendation rules and what-if scenarios

3.2 Big Data Architecture

Raw CSV is ingested with explicit declared schemas rather than by inference. Inference is convenient and wrong here: an integer column with one null in the sample becomes a double, a date becomes a string, and a drifted column type is accepted silently and then quietly mis-analyzed. A declared schema fails at ingestion, which is the only place the failure is cheap. The specification also asks for inference to be

demonstrated, so the ingestion stage reports a declared-versus-inferred comparison alongside the load it actually performs.

Storage is Parquet, partitioned by month for the tables that grow with the length of the observation window: delays, passenger_counts, tickets, trips. Partitioning is chosen on that basis rather than applied everywhere, because a partition column on a table of a few hundred rows adds directories and metadata without removing any work. The benefit on the large tables is that most downstream reads are bounded by a date range, so the planner touches the partitions the query needs and skips the rest, and the columnar format then reads only the columns each aggregation names

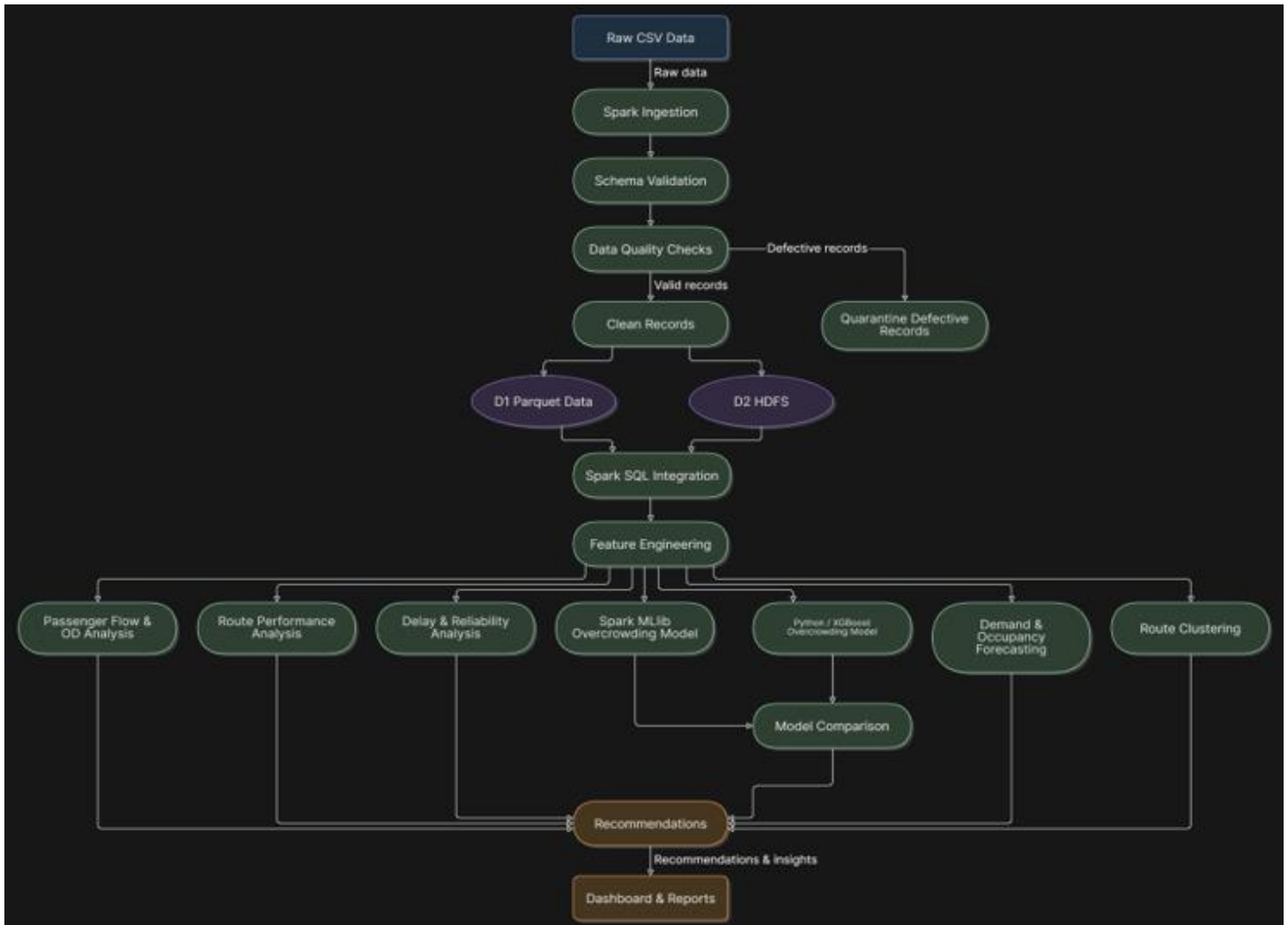
HDFS is the distributed storage for the processed layer and it operates on one node; the processed Parquet tree gets stored on it and then read from it and verified instead of just making assumptions.

3.3 Data Flow Diagram

Data Flow Diagram (Level 1)



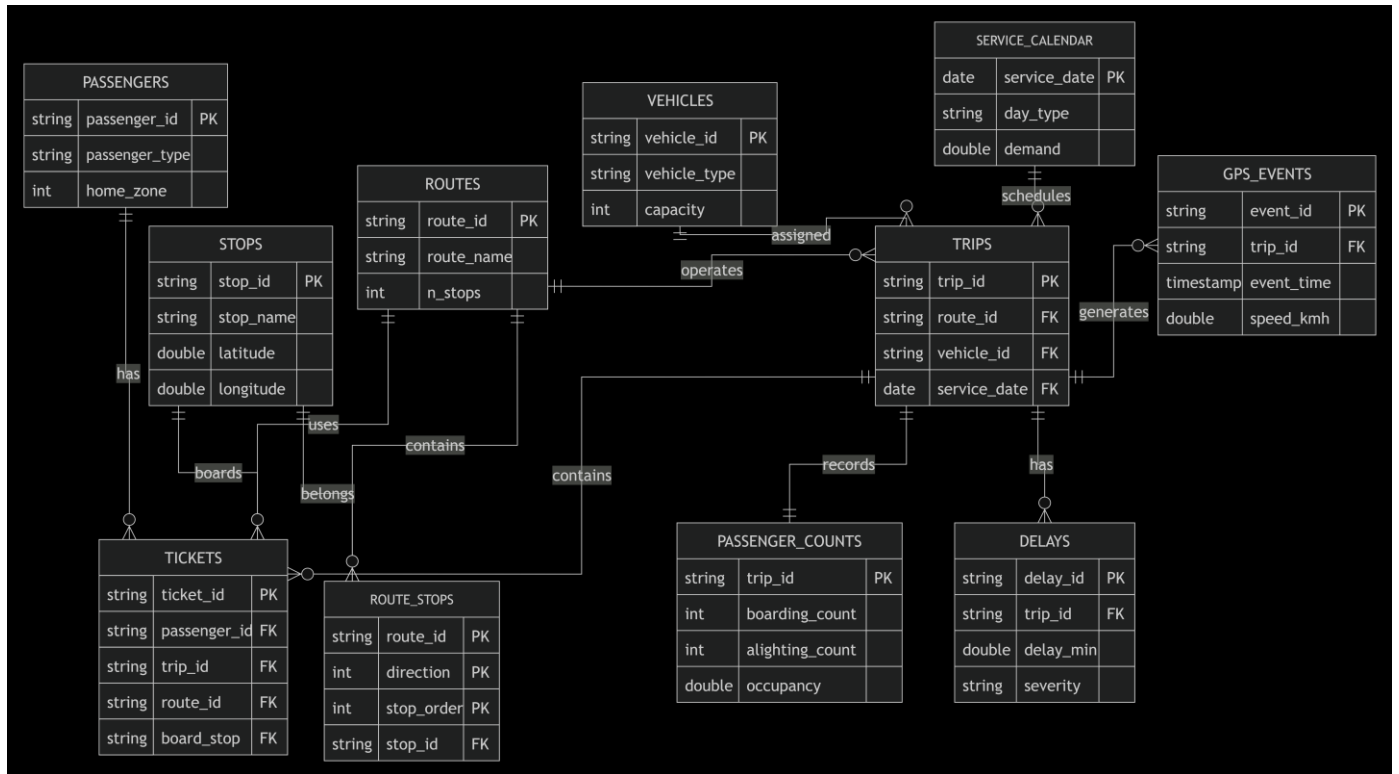
Data Flow Diagram (Level2)



4. Database Design

The tables are connected, representing one year of operations, via primary and foreign keys. The design is partitioned in such a way that the database distinguishes between what the network is and what it does: the stops, routes, route_stops, vehicles, schedules and service calendar represent a schema that changes rarely, whereas trips, passenger_counts, delays, tickets and gps_events represent events happening to the schema on daily basis. Any analysis in the project involves joining the latter onto the former, and that is why the integration phase results in only one trip-level fact table.

Entity Relationship Diagram



Entity Relationships

Child table	Column	Parent table
route_stops	route_id	routes
route_stops	stop_id	stops
schedules	route_id	routes
trips	route_id	routes
trips	vehicle_id	vehicles
passenger_counts	trip_id	trips
passenger_counts	route_id	routes
delays	trip_id	trips
delays	route_id	routes
tickets	passenger_id	passengers
tickets	trip_id	trips
tickets	route_id	routes
tickets	board_stop	stops
tickets	alight_stop	stops
gps_events	trip_id	trips

4.1 Data Dictionary

Table	Rows	Columns	Primary key	Partitioned by
stops	520	7	stop_id	-
routes	100	10	route_id	-
route_stops	3,068	5	route_id, direction, stop_order	-
vehicles	260	5	vehicle_id	-
service_calendar	365	4	service_date	-
schedules	900	7	service_id	-
passengers	50,000	4	passenger_id	-
trips	567,248	14	trip_id	month
passenger_counts	567,248	13	trip_id	month
delays	305,537	12	delay_id	month
tickets	2,012,000	10	ticket_id	month
gps_events	800,344	6	-	-

4.3 Dataset Profile

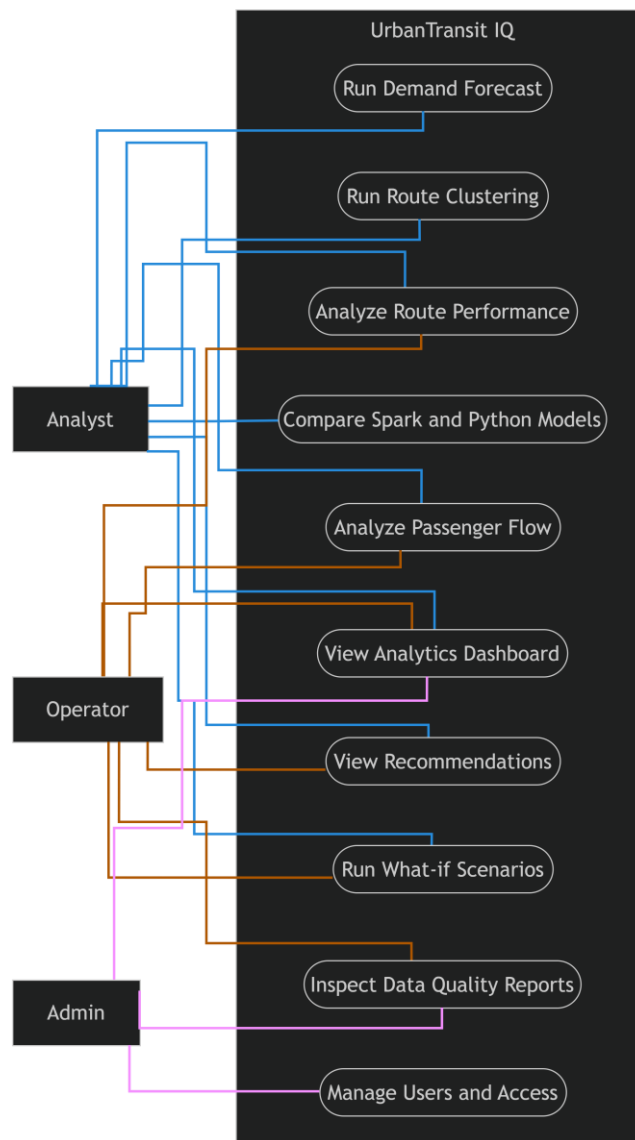
In addition, the catalog phase creates a statistical profile of per-column minimum and maximum ranges, means and standard deviations, first and last values for temporal columns, and exact distinct counts and distributions of values. The need for exact distinct counts may seem to be pedantic but is not: the approximate distinct count came up with more vehicles and less stops than there were.

Table	Rows	Columns	Numeric profiled
stops	520	7	3
routes	100	10	6
route_stops	3,068	5	3
vehicles	260	5	2
service_calendar	365	4	1
schedules	900	7	2
passengers	50,000	4	1
trips	567,248	14	5
passenger_counts	567,248	13	9
delays	305,537	12	3
tickets	2,012,000	10	2
gps_events	800,344	6	3

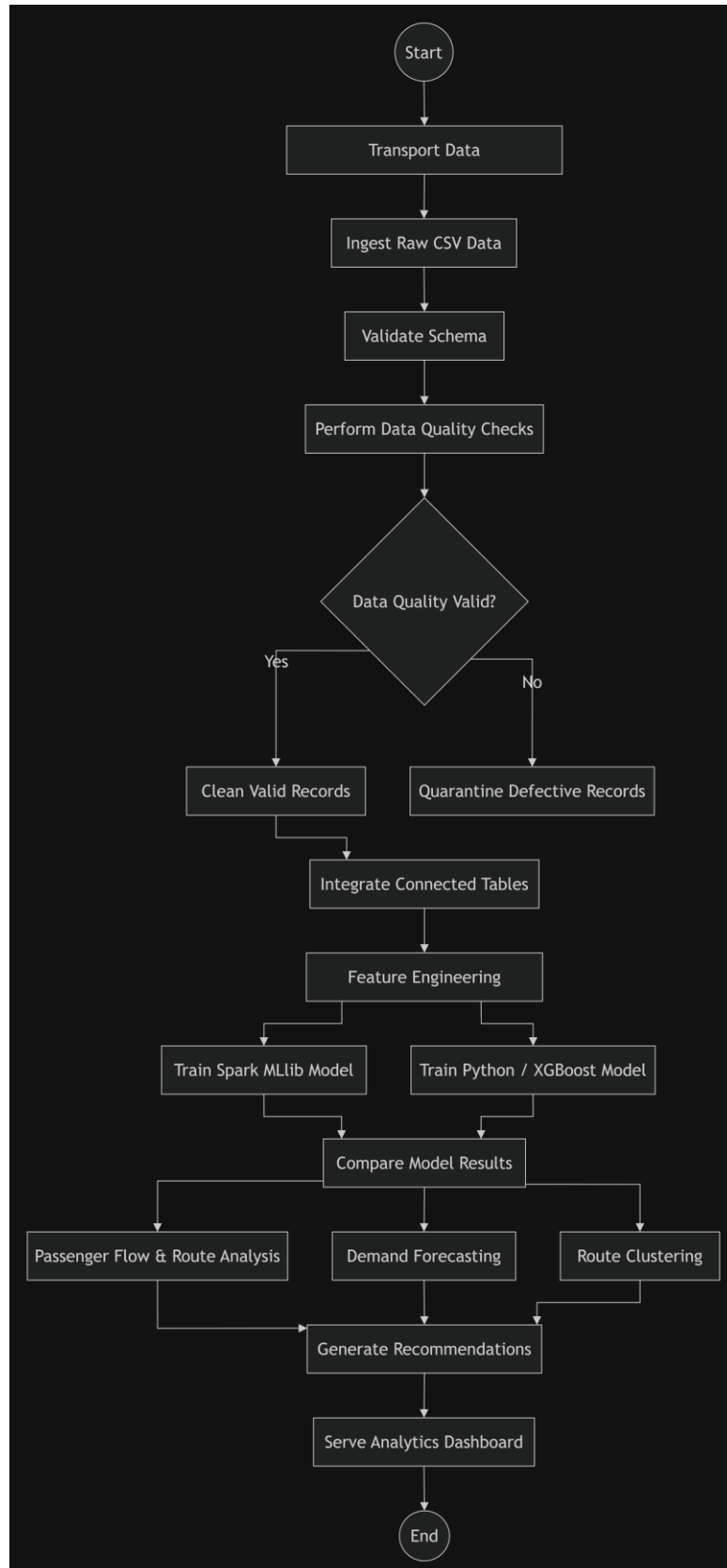
5. Behavioural Models

5.1 Use-Case Diagram

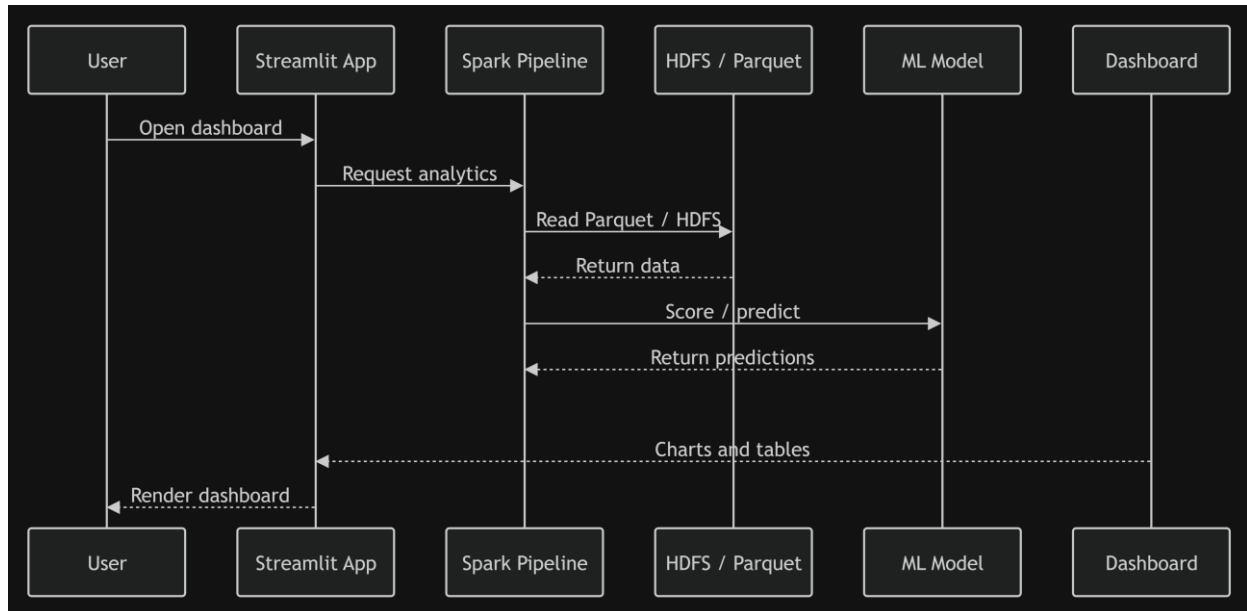
Role	What they see
Transit Operator	Operates the transit system on a day-to-day basis: what's crowded,
Data Analyst	Learns the analysis and the theory behind it, then exports the graphs.
Network Admin	All that an evaluator knows, plus the accounts themselves.



5.2 Activity Diagram



5.3 Sequence Diagram



6. Methodology

6.1 Dataset Generation

The seeded simulator generates all twelve tables with demand based on the hour of the day, day of the week, seasonal variations, and per-route personality, along with intentional data quality flaws (ID missing, duplicates, bad timestamps, negative counts), to be detected by the data quality module.

Since there is no such publicly available dataset resembling a year's worth of operation of a mid-size network with all its mess intact, a new one is created. The key feature of the data generator is that it does not generate random numbers but rather structure, since the structure is what the analytics have to recreate. Each route receives a base demand, a peak bias, a directional bias, a reliability bias, and a weekend bias, and these are the parameters which make one route a congested commuting route and another one a nearly empty suburban circle.

The demand is generated based on a weight curve from 0.05 at 0:00 AM to 3 at 08:00 AM, combined with a day type, monthly and route specific multiplier, while the boardings will be sampled with the calculated rate. Analytics manage to pick up that shape without even being told about it – the most crowded departure hour based on ticket information is 08:00 with 4,670,516 boardings.

Measured demand by day type, which is the shape of the generator and the one that has to be recovered by analytics based on ticket flows only:

Day type	Boardings	Average per route-day
weekday	21,111,388	842.8
weekend	3,636,971	363.0
holiday	171,625	218.4

The fleet is mixed, because a single capacity would make occupancy a restatement of demand and would remove the vehicle-allocation question entirely:

Vehicle class	Seated capacity	Share of fleet
Minibus	30	25%
Standard Bus	55	50%
Articulated Bus	90	25%

There are ten named events on the calendar, eight of which increase demand and two of which decrease demand. Both are by design. A detection system that only detects spikes is unable to detect days where a network loses half its ridership because of the weather or road closures. Such days need to be detected just as much as the concert day. Prior to the creation of these events, the spike detector was detecting weekday spikes in December and labeling them events, which was no detection at all.

Date	Event	Demand multiplier
2025-02-14	Festival	1.9x
2025-03-22	sports match	2.4x
2025-05-01	public rally	2.1x
2025-06-18	weather disruption	0.45x
2025-07-05	Concert	2.2x
2025-08-23	sports match	2.3x
2025-09-13	Festival	2x
2025-10-11	marathon road closure	0.55x
2025-11-08	Concert	2.5x
2025-12-20	shopping weekend	1.8x

Delay generation is the portion of the simulator that was redeveloped, and the reasoning behind it should be noted. It started out as one large normal draw, and as such, delay was nearly noise as the severity classifier could not reach even the low fifties in accuracy as there was nothing to learn from the data. Delay is now constructed from an hourly congestion profile where the peak occurs at 6.7 minutes between 17:00 hours, a day-of-week variable, a seasonality variable, the reliability of the route itself, total dwell time at bottleneck stops, a random component with a standard deviation of 1.2 minutes, and incident conditions that occur 0.4 percent of the time on average 15 minutes long. Seasonality variable runs from 0.2 to 1.7 over the year period, and it is the most important variable in the entire generator for modeling purposes. Detection can be validated against the expected amount:

Injected defect	Rate
missing passenger id	1.000%
missing route id	0.400%
duplicate ticket	0.600%
invalid timestamp	0.300%
negative passenger count	0.200%
capacity violation	1.500%
invalid delay	0.200%
unknown passenger	0.100%
trip cancellation	2.000%

6.2 Data Quality & Cleaning

The set of 41 tests spans all the defect classes listed in the specification, including missing ticket instances, missing/unresolved route/stop/trip/vehicle/passenger ids, duplicate tickets/trips, negative values, invalid date/time values, impossible arrival times, departure before arrival, capacity violations, invalid delays and distances, and invalid stop sequences. Errors are fixed or isolated with an audit trail that captures the original value, the rule used, the new value, and the status. The biggest findings are:

Table	Issue	Records
passenger_counts	over_capacity	100,347
trips	no_ticket_records	73,731
tickets	missing_passenger_id	19,967
tickets	duplicate_primary_key	12,000
passenger_counts	implausible_load	8,986
tickets	missing_route_id	8,197
tickets	board_time_out_of_range	6,036
delays	impossible_arrival_time	2,161

- Duplicate tickets are deleted; routes without IDs are quarantined.
- Invalid (future) time stamps and negative counts are quarantined.
- Overloading (number of passengers exceeding capacity) is retained and marked; impossible values are deleted.

One-third of the tests are hand-coded, while the remaining two-thirds are derived from the schema registry because the registry specifies the keys and, by definition, provides a uniqueness test and a referential test

for each. Hand-coded rules are the semantic layer, where a value cannot be inferred by the structure: a negative fare, a boarding time stamp in a year before the existence of the network, an active trip without ticket movements.

The largest finding by far is the best example. There are 100,347 rows with onboard loads higher than the seated capacity, and this finding is annotated info and retained because it is not a defect. It is overcrowding, which is what the project is meant to identify. Cleaning removes loads that are more than three times the capacity, which is certainly not a bus but a defective sensor, and retains loads over capacity, which are the answer. There is a test to verify that the second set of records survived cleaning, since any benevolent attempt to trim the occupancy figure down to 100% will remove the main finding of the project and make all other numbers look better.

The validity boundaries are single-sourced for the same reason. There are no timestamps beyond 2026, and delays greater than 300 minutes are not delays. Both limits used to exist twice before, once in the validity check and once in the cleaning operation, so changing one of them would result in detecting against one limit and cleaning against another.

Handling has three verbs, and these are captured by the audit trail on the number of records that have been handled along with the value changed and the correct value.

Action	Records affected
quarantine	24,833
correct	21,690
dedup	12,000

What is worth noting is that detection took place before management of those detected. The ticket will refer to a passenger who is not listed in the passengers table. This check was performed on all of them, reporting correctly without any follow-up to correct them; therefore, the number of headline passengers was higher than the number of registered passengers. Unknown cards will now be linked to the same sentinel nulls that have been used before for the reason that the journey is evidence enough.

6.3 Spark Processing & Features

The data gets ingested with schemas, joined with Spark SQL to create a trip level fact table and used to generate features in trip, route-day, route and stop grains (headway, load factor, punctuality, occupancy, reliability, bunching).

There are four different grains because there are four different levels where the questions exist, and blending those will create nonsense. The trip row is that which the overcrowding and delay models will be predicting

and may contain only those features which are known before the vehicle departs. The route-day row is the one on which the demand forecast and persistence decision will take place since a route that is overcrowded most days needs a different treatment than the one overcrowded only for three days. The route row will contain the composite score and the classification while the stop row will have boardings and alighting.

Trip-grain population is the one that requires caution. The fact table contains 555,891 trips that were executed out of 567,248 trips that were scheduled in the source. The discrepancy here refers to trips that were scheduled but not executed, and therefore are not included in average occupancy and delays but rather are considered as missed trips, as there was no occupancy and punctuality for them at all.

6.4 Route Performance Scoring

Each path gets an aggregate score between 0 and 100 based on a number of components which are assigned different weights, making it possible to rank routes instead of just classifying them. These weights are stored in the settings.py file, while a component is printed along with the total score.

Component	Weight
Utilisation	25%
Punctuality	20%
Delay	15%
Crowding	15%
Demand	10%
delay frequency	8%
travel time	7%

Two of the factors require a benchmark, and both are clearly defined. The utilisation factor is based on a target level of occupancy of 70% rather than a maximum because a bus route that operates at its maximum capacity is not operating effectively; it is a route with no spare capacity, and the next problem that occurs causes passengers to be stranded. Utilisation based on a factor of more is better would rate the busiest and most crowded routes in the network as its best performers. Delay time is scored against a limit of 15 minutes, after which the component becomes zero because the difference between a delay of 20 and 40 minutes is not something that can be distinguished in triage.

Rank	Route	Score	Occupancy	On-time	Delay (min)	Classification
1	R016	75.1	77%	59%	4.44	High Performing
2	R023	73.6	69%	45%	5.24	High Performing
3	R030	73.2	62%	60%	4.38	High Performing
4	R072	71.2	81%	49%	4.89	High Performing
5	R025	71.0	82%	45%	5.2	High Performing
6	R039	70.6	73%	34%	6.04	High Performing
7	R014	69.9	77%	38%	5.72	High Performing
8	R077	69.0	49%	54%	4.78	High Performing
9	R065	68.6	83%	35%	6.07	High Performing
10	R015	68.5	45%	59%	4.5	High Performing

This is a range that begins with 75.1 at the top end and goes down to 42.3 for R006 at the lower end.

Classification is independent of the score and represents a different inquiry, namely, what kind of issue this particular route presents. Five different types of classifications are awarded based on occupancy, reliability, and demand separately, since two routes with identical scores may require different treatment.

Category	Routes
High Performing	42
High Demand, Unreliable	24
Reliable, Underutilized	19
Low Performing	10
Overcrowded	5

The punctuality threshold behind those categories is 5 minutes, and the number was changed once for a reason worth recording. A stricter two-minute window, applied after delay generation was rebuilt, compressed every route's on-time rate into a band of about two percentage points. The classifier then labelled all 100 routes unreliable, one category emptied completely, and the recommendation rule that depends on it stopped producing advice. The fault was not the delay model but the definition: two minutes is far stricter than the window operators publish against. At 5 minutes the measured rates spread from 0.032 to 0.601 and all five categories are populated. The late threshold is now derived from the on-time threshold rather than being a second number that could drift away from it.

One other correction applies to all reliability figures presented here. Punctuality was initially calculated on the basis of either a delay up to the threshold or no delay at all, so each unmeasured journey counted as

punctual, thus making the published figure rather high. The number of trips with a delay is small, and so the new rate is now based on trips measured, with the latter printed alongside.

Reliability figures are published for each route along with their variations and bases, and so one figure based on a few hundred trips is not treated as being based on thousands of trips:

route id	on time rate	delay variation	travel consistency	time early arrivals	trips operated	scheduled trips	missed trips	missed rate
R030	0.601	2.32	0.07	18	5156	5282	126	0.0239
R015	0.594	2.43	0.073	23	6256	6378	122	0.0191
R016	0.59	2.49	0.03	33	6843	6982	139	0.0199
R059	0.569	2.42	0.108	14	4106	4208	102	0.0242
R057	0.549	2.69	0.045	29	4841	4932	91	0.0185
R077	0.539	2.69	0.085	39	6749	6886	137	0.0199
R080	0.524	2.86	0.098	26	4677	4764	87	0.0183
R035	0.518	2.39	0.065	20	4765	4850	85	0.0175
R010	0.508	2.26	0.026	4	3086	3152	66	0.0209
R081	0.505	2.67	0.111	18	3861	3950	89	0.0225

6.5 Bottleneck Detection

A bottleneck is known through behavior, not through an identifying tag. Each one is scored for mean delay time on the routes that use it, percentage overcrowding, net gain in passengers, turnover, boarding concentration during peak times, and number of connecting routes.

Signal	Weight
Delay	30%
Crowding	22%
Accumulation	14%
Turnover	14%
Peaking	12%
Connectivity	8%

The threshold is set at the 97th percentile of the score distribution and not at a constant number or score so that regardless of how large the network is, the same code generates a sensible short list: a fixed number will generate the same number of stops from networks of fifty stops as networks of five thousand.

This section of code replaced an earlier version that wasn't only incomplete but fundamentally flawed. This filter only detected the presence of the stop-type column, which is the label the generator placed by itself. On this data set, it generated precisely the planted stops and appeared to work perfectly; on a separate hidden data set where this column did not exist, it would return no stops at all. The current detector never accesses that column.

Stop	Score	Avg delay	Crowding	Accumulation	Routes
S0141	0.661	7.23	24%	3,848	8
S0374	0.639	8.65	33%	269	6
S0065	0.616	8.7	21%	3,555	5
S0365	0.614	11.59	29%	657	1
S0125	0.603	7.87	30%	3,026	4
S0441	0.6	8.14	25%	1,379	7
S0016	0.598	9.55	27%	1,809	2
S0372	0.597	7.86	31%	1,352	4

The detector identifies 19 stops (3% of stops sorted by score). Of the stops identified by the generator, 8 stops are identified as bottlenecks and 1 of these bottlenecks is identified, resulting in a recall of 0.125. The intersection of these two values is an incomplete set by design, and the reason can be seen in the two numbers

provided by the generator: the bottleneck module introduces an additional 1 minute of dwell for each stop that is marked as a bottleneck, whereas the congestion profile delays delay up to 6.7 minutes.

Low overlap does not prove the detector's efficacy, but is rather a proof that they have two different meanings, which is clearly specified here instead of hiding somewhere. While the following statement is true, that the detector ranks stops according to the observed delays and crowding, and provides the reasons for each ranking without any use of the column a real data would never contain. The only question that still remains open is whether the list provided is the correct one operationally speaking.

6.6 Analytics

- Route-based score and multi-criteria classification.
- Peak hour, passenger flow and O-D (origin-destination) study.
- Delay, scheduling performance, service frequency and gap between supply and demand.
- Special event, anomaly detection, and passenger classification.

The network level series is the framework for interpretation for all route level conclusions drawn, and occupancy, punctuality and delays do not co-vary each month:

Month	Trips	Overcrowded	Avg occupancy	On-time rate	Avg delay
1	47,301	7,881	0.462	0.218	7.88
2	43,068	7,436	0.475	0.244	7.63
3	45,792	8,558	0.497	0.285	7.16
4	45,535	9,109	0.523	0.323	6.87
5	46,689	9,972	0.548	0.315	6.92
6	38,900	6,700	0.473	0.327	6.82

Month	Trips	Overcrowded	Avg occupancy	On-time rate	Avg delay
7	41,194	5,963	0.431	0.345	6.64
8	39,293	5,493	0.415	0.342	6.67
9	51,267	11,207	0.563	0.3	7.09
10	53,223	12,001	0.569	0.251	7.55
11	50,331	10,889	0.556	0.234	7.67
12	53,298	12,622	0.598	0.179	8.3

Most congested days, which are the days on which a peak capacity schedule must operate successfully:

Date	Boardings
2025-02-14	120,042
2025-12-16	110,507
2025-12-10	110,251
2025-12-30	110,128
2025-12-15	110,117
2025-12-31	109,323
2025-12-08	108,684
2025-12-05	107,812

Occupancies are grouped rather than averaged, since an average for a bimodal network does not describe either mode. The groupings are based on configuration, and the frequencies are measured as trips:

Band	Occupancy range	Trips
Low	0 to 0.35	255,267
Moderate	0.35 to 0.7	140,121
High	0.7 to 0.9	42,595
Overcrowded	0.9 to 1	16,657
Critical	above 1	91,174

6.7 Spark SQL

This is performed using one Spark SQL query against the temporary views generated over the cleaned Parquet. The query is fully given below since the assignment specifies that queries be included rather than their description.

6.8 Passenger Flow

Entries and exits are tallied per stop in five parts of the day, so a stop which gets filled in the morning and emptied in the evening can be differentiated from a stop which operates busily throughout the day.

Stop	Boardings	Alightings	Turnover	Net accumulation	Peak hour	Routes
S0141	15,809	11,961	27,770	3,848	18:00	8
S0502	8,363	15,754	24,117	-7,391	18:00	6
S0404	8,410	11,759	20,169	-3,349	08:00	5
S0171	9,851	10,236	20,087	-385	18:00	5
S0245	6,047	14,008	20,055	-7,961	18:00	5
S0173	7,639	11,866	19,505	-4,227	18:00	7
S0351	5,421	13,547	18,968	-8,126	18:00	4
S0484	6,558	12,327	18,885	-5,769	08:00	5
S0322	5,079	13,476	18,555	-8,397	08:00	4
S0436	8,646	9,759	18,405	-1,113	18:00	4

The net accumulation is the busiest column and the most misleading one among all the columns in terms of interpretation. This is simply entries less than exits for the entire period, hence a large positive number represents a net source while a large negative number is a net destination. This is neither a measure of queue length nor the number of people standing on a stop waiting to board; it is a balance of directionality throughout the year, which in the case of a large figure, indicates structural one-way service. This is a scheduling issue rather than crowding.

Demand for a certain route does not operate symmetrically in both directions:

Route	Direction	Passengers	Occupancy	Network share
R066	1	311,003	90%	1.25%
R066	0	310,924	90%	1.25%
R090	1	290,736	92%	1.17%
R028	1	274,987	92%	1.10%
R090	0	274,956	86%	1.10%
R063	1	273,161	81%	1.10%
R063	0	272,697	81%	1.09%
R002	1	269,142	93%	1.08%

Day of week	Boardings
1	1,757,140
2	4,278,063
3	4,249,535
4	4,278,733
5	4,104,108
6	4,337,172
7	1,915,233

6.9 Origin-Destination Analysis

The matrix contains 149,178 unique combinations for origin, destination, route, time period and day type. The largest flows include:

From	To	Route	Period	Day type	Passengers
S0388	S0404	R039	am_peak	Weekday	623
S0436	S0404	R039	am_peak	Weekday	585
S0409	S0317	R025	am_peak	Weekday	574
S0467	S0469	R014	am_peak	Weekday	567
S0148	S0137	R025	am_peak	Weekday	563
S0401	S0351	R039	am_peak	Weekday	562
S0388	S0404	R039	pm_peak	Weekday	555
S0078	S0137	R025	pm_peak	Weekday	547
S0083	S0317	R025	am_peak	Weekday	546
S0436	S0404	R039	pm_peak	Weekday	545

Time period	Passengers
pm_peak	728,295
am_peak	772,473
Evening	241,901
Midday	204,860
Early	38,349

Day type	Passengers
weekday	1,565,316
weekend	397,521
holiday	23,041

6.10 Delay Analysis

Delay has been separated according to all ways prescribed in the specification: by route, trip, stop, vehicle, time of day, day of week, direction, and distance travelled.

Bands of severity have been defined in configuration and have been presented as counts. The distribution is what makes the classification task of 7.3 more difficult than implied by its accuracy metric.:

Band	Delay range (min)	Records
On Time	0 to 2	11,332
Minor Delay	2 to 5	67,881
Moderate Delay	5 to 10	155,884
Major Delay	10 to 20	69,263
Severe Delay	above 20	566

There are 155,884 observations of Moderate Delay and 566 observations of Severe Delay. It's the ratio like this, where the smallest class is to the largest class, that makes macro-averaging a difficult metric because a classification algorithm can get almost perfect scores for its predictions by not predicting the smallest class at all.

By stop

stop id	avg delay	trips	routes
S0365	11.59	2997	1
S0208	10.48	3115	1
S0175	10.48	3115	1
S0457	10.13	2584	1
S0295	9.84	2743	1
S0115	9.84	3442	1
S0196	9.81	5930	2
S0419	9.81	3633	1

By vehicle

vehicle id	vehicle type	avg delay	trips
V0100	Standard Bus	9.04	1482
V0056	Articulated Bus	8.71	1526
V0129	Standard Bus	8.62	1536
V0199	Articulated Bus	8.52	1578
V0251	Minibus	8.51	1584
V0157	Articulated Bus	8.45	1545
V0077	Standard Bus	8.42	1585
V0085	Standard Bus	8.37	1577

By direction

direction	avg delay	trips
1	7.6	152286
0	7.34	152640

By day of week

dow	avg delay	trips
1	6.39	30940
2	7.91	48410
3	7.34	48405
4	7.47	49536
5	7.67	47333
6	8.21	49083
7	6.59	31219

By distance

band	avg delay	avg km	trips
16-24 km	7.68	20.0	100276
over 24 km	7.77	28.2	69864
under 8 km	6.32	7.5	13199
8-16 km	7.25	11.7	121587

Two of these breakdowns were initially analytically correct but entirely void, and the solution was to be found not in the analysis but in the generator. The delay by vehicle ranged from a tenth of a minute in the worst vehicles of the fleet, while delay by direction showed the exact same range in both directions, since neither vehicle nor direction possessed any trait that could create the discrepancy. Vehicles currently have a condition of up to 1.4 minutes, while directions have an imbalance of up to 0.9 minutes, and the delay by vehicle ranges from 6.26 to 9.04 minutes. Analyzing correctly data that lacks all substance is a failure mode that needs its name, because it creates void tables that pass all review.

Patterns in routes and times would be the places where a new schedule would be useful:

Route	Hour	Average delay	Trips
R021	17:00	16.3	278
R021	08:00	15.6	303
R064	17:00	14.7	321
R021	18:00	14.6	328
R071	17:00	14.5	261
R034	17:00	14.5	313
R089	17:00	14.4	276
R004	17:00	14.3	355
R021	07:00	14.2	278

Combination only counts as a pattern if there are at least 40 trips behind it. Without a cut-off, the table would just be filled with route-hours that had a few runs and an accident, which would not be considered a recurrent problem. The cut-off means the difference between a table from which you can adjust your schedule and a table of accidents.

Schedule adherence, by type of run:

Adherence	Trips
Late	155,884
On time	77,967
Very late	69,829
Early	1,246

6.11 Anomaly Detection

Ten types of strange behavior are being searched for. The numbers are the number of records identified, not an estimation.

Anomaly	Records
occupancy outliers	3,799
extreme delays	44
demand spikes	444
demand drops	11
near empty trips	24,438
unexpected route usage	0

Anomaly	Records
abnormal travel times	26
irregular stop activity	20
duplicate ticketing	see data_quality_report.csv, duplicate_primary_key
impossible occupancy	see data_quality_report.csv, implausible_load

Amongst the ten detectors, one shows nothing in relation to this dataset (irregular routing pattern) and that is shown as zero, not tuned down until it gives any result. A detector whose tuning goes down until it generates a signal is no longer a detector and a zero is information regarding the dataset which the reader can use to know that there is no example of that particular pattern on this dataset at the level of threshold that the project has set ahead of time.

The thresholds used in these counts are fixed and published. An outlier of occupancy would be one that is beyond 3.5 standard deviation, travel time would also be one that is beyond 3.5 standard deviation, and an extremely late delay would be any value above 60 minutes, and this is an absolute rule because being late by an hour means just that without regard to variation of the road.

The high demand days are determined using each individual route's baseline for the same type of day and not the baseline of the whole network, which means that an inactive route will be equally captured as a highly active one if there is any deviation. 444 route-days have more than 3 standard deviations away from the baseline.

route id	service date	day type	demand	typical for day type	z
R007	2025-05-01	holiday	105	42	42.5
R042	2025-05-01	holiday	487	89	26.9
R000	2025-05-01	holiday	1019	346	22.7
R021	2025-05-01	holiday	190	91	22.3
R092	2025-05-01	holiday	560	154	21.1
R091	2025-05-01	holiday	79	18	20.6
R022	2025-05-01	holiday	279	88	18.4
R081	2025-05-01	holiday	115	35	18.0

Event detection is evaluated with respect to the calendar provided to the generator: 10 planted events, 9 found, recall = 0.90. Events are never seen by the detector; recall less than 1 should not surprise but rather reassure us that a day with an event multiplier lying within the range of normal days of that type cannot be separated from normal days, and so the experiment is reported thusly so that any future data set of subtle events will yield an honest result.

Date	Event kind
2025-02-14	festival
2025-03-22	sports match
2025-05-01	public rally
2025-06-18	weather disruption
2025-07-05	concert
2025-08-23	sports match
2025-09-13	festival
2025-11-08	concert
2025-12-20	shopping weekend

Falls in demand are also registered along with rises, hence the two-tailed test. Largest falls in demand on the network:

route id	service date	day type	demand	typical	usual swing	z
R021	2025-04-01	holiday	54	91	3	-8.32
R021	2025-12-25	holiday	57	91	3	-7.64
R003	2025-06-07	holiday	216	455	23	-7.01
R003	2025-08-14	holiday	255	455	23	-5.87
R099	2025-12-07	weekend	425	735	56	-3.73
R069	2025-01-01	holiday	60	163	19	-3.66
R099	2025-11-23	weekend	450	735	56	-3.43
R085	2025-01-01	holiday	44	118	15	-3.33

Passengers are categorized on the basis of behavior rather than according to what the generator put into the type column, and such categorization is more of a control on the generator than an observation in itself, since two of these categories remained always empty when ticket allocation was even because the passengers did not matter at all.

segment	count	avg trips	avg km	trip peak share	weekend share	preferred routes	distinct od pairs
Daily Commuter	24968	70.2	17.9	0.669	0.188	49.2	69.7
Weekend Traveller	5989	10.7	17.1	0.534	0.596	9.9	10.7
Long-Distance Traveller	5227	8.8	23.4	0.533	0.204	8.0	8.8
Occasional Traveller	4726	3.1	16.4	0.319	0.099	3.1	3.1
Peak-Hour Traveller	4606	7.0	16.8	0.806	0.156	6.5	7.0
Regular Traveller	3635	15.0	18.5	0.463	0.27	13.8	15.0

6.12 Travel Time, Scheduled Against Actual

Every delay event involves both a recorded departure and a recorded arrival, so that the actual run time becomes a measured value, not the schedule time plus the departure delay. This is important because a vehicle can depart on schedule and yet lose ten minutes during the run. On-peak and off-peak run times are differentiated, and each route is compared to itself historically.

Route	Scheduled	Actual	Gap	Peak	Off-peak	Peak penalty	vs earlier months
R010	119.4	133.6	14.2	140.9	128.7	12.2	0.4
R070	108.8	122.8	14.0	129.6	118.6	11.0	0.4
R013	70.2	83.2	13.0	87.2	80.7	6.5	-0.1
R090	85.4	98.4	13.0	103.0	95.2	7.8	0.1
R016	100.5	113.3	12.8	119.4	109.1	10.3	-0.3
R092	88.1	99.2	11.1	104.8	95.4	9.4	-0.1
R061	99.9	110.0	10.1	116.5	105.8	10.7	-0.3
R031	75.2	84.4	9.2	88.2	81.8	6.4	-0.1
R011	74.6	83.7	9.1	88.6	80.5	8.1	0.2
R029	86.2	95.0	8.8	100.1	91.5	8.6	0.3

6.13 Headway, Bunching and Service Regularity

The headway is the time interval between successive departures from a route and direction, and this is the real value experienced by waiting passengers. Two types of problems are searched for in contrasting ways: vehicles that depart too close together, resulting in wasted capacity and a subsequent gap being too long; and gaps much larger than the route's normal, where waiting time is lost.

Measure	Value
Average headway (min)	120.6
Median headway (min)	79.0
Bunched trips	12,277
Excessive gaps	97,651
Irregular routes	12
Irregularity cutoff (CV)	1.106
Excessive-gap rule	over 2.5x the route's own median

For bunching, the definition uses an absolute threshold of 3 minutes, since two cars leaving so close is always bunching, no matter what the network. For gap and irregularity, the definitions are relative for the reverse reason; both started out as absolute guesses, but neither made sense given the service pattern of the system, where the median headway is 79 minutes, meaning that any rule of 45 minutes or more would flag almost all trips as having an excessive gap, and departures being drawn hour weighted means the coefficient of variation will be roughly 1 for all routes, so any absolute definition would flag all routes as being irregular. Excessive gaps and irregularity are now defined as 2.5x and 1.106x respectively of the network's own distribution.

Bunching is calculated using two modules, and both modules output 12,277. The modules did not match at first since one calculates using strictly positive headways only, thus neglecting all trips where the headway was equal to zero, which is the worst case of bunching, i.e., two buses leaving in one minute. A regression test proves the values match each other, since if a number is repeated in a report, this number can be drifted.

Per route, in descending order of how much the actual headway deviates from the planned:

route id	median	planned	headway	vs headway	bunch	excessive gaps	is irregular
R023	59.0	91.7	1.074	1.178	211	1255	1
R004	60.0	95.4	1.027	1.163	175	1142	1
R021	67.0	115.4	0.947	1.141	144	983	1
R069	64.0	101.2	1.018	1.137	157	1076	1

route id	median headway	planned headway	headway plan	vs headway cv	bunch ed	excessive gaps	is irregular
R043	72.0	115.8	0.999	1.13	123	859	1
R020	61.0	98.9	1.036	1.126	165	1206	1
R072	62.0	95.1	1.075	1.125	185	1210	1
R053	63.0	95.0	1.079	1.123	159	1149	1
R018	62.0	93.3	1.106	1.118	157	1230	1
R054	65.0	95.4	1.078	1.116	149	1070	1

6.14 Service Frequency and the Demand-Supply Gap

Both judgements are performed per route and both are outputted per route rather than as a distribution only. Frequency is concerned whether the trip count corresponds to the demand pattern, including the share of the demand that is realized during the peak hours. Demand-supply difference poses the more direct question whether the seat capacity provided throughout the year is sufficient for the passengers carried.

Frequency verdict	Routes
Well matched	71
Too much frequency	20
Too little frequency	9

Demand-supply status	Routes
Balanced	56
Excess demand	32
Excess supply	12

Both of these indicators were once exported as a count per judgement with the route IDs stripped off, which makes them useless for their purpose. Distribution showing an operator the number of routes that have inadequate service provision does not show which routes they are. The route-level data is now the artifact while the summary is the result of its analysis.

The most unbalanced routes in each category:

route id	demand	supply	gap ratio	status
R002	525325	415515	1.264	Excess demand
R066	621927	499320	1.246	Excess demand
R028	534300	430510	1.241	Excess demand
R090	565692	458490	1.234	Excess demand
R067	490114	400720	1.223	Excess demand
R091	34741	182030	0.191	Excess supply
R008	37242	197930	0.188	Excess supply
R006	30907	170060	0.182	Excess supply
R007	38497	228180	0.169	Excess supply

The bands are above ninety percent of the supply and represent excess demand, while they are below thirty-five percent and represent excess supply, with the middle one referred to as balanced. The upper band is deliberately set below the parity: it represents a route that is already turning people away during the peak times since there is uneven demand in the course of a day and the ratio for the year is an average through the off-times.

7. Machine-Learning Models

7.1 How the Data Is Split

All models used in this project use the exact dates for the cut and cut the information by time, and not randomly. The first half of the months fits into the model, the middle month picks the winner algorithm, and the last three months receive only one score, which becomes the reported number. Selecting the winning algorithm on the test set would mean creating another validation set and inflating the main score.

Part	Months	Trips	Used for
train	1-8	347,772	fitting
validation	9	51,267	model selection
test	10-12	156,852	the reported numbers, scored once

The boundaries are 2025-09-01 and 2025-10-01, the boundaries are stored in the configuration module, and the three model modules take boundaries from there. It is intentional since the split was done in two ways initially, and each model module had its own dates, so the boundaries might become different, and then it wouldn't be possible to compare the results of two models. One place gives the basis for comparison in 7.2.

The random split enables learning on trips that are happening after the trips used for scoring. All the models in the project split in the same way to make numbers comparable.

7.2 Dual Pipeline , Overcrowding Prediction

Overcrowding will be predicted using pre-trip features only (without any leakage). Overcrowding is computed separately in Spark MLlib and in Python, with each platform choosing the algorithm in the validation month, followed by scoring in the test quarter.

The feature contract is the only shared item between the two pipelines: 3 categorical and 12 numeric features, and all of them are known prior to the trip. This constraint is not assumed but enforced. The outcome columns are found right next to the input columns in the feature table, which means that the leak guard will fail when an outcome column gets into the feature table. Categorical: route_id, vehicle_type, day_type. Numeric: direction, dep_hour, capacity, route_reliability, demand_mult, planned_min, route_km, n_stops, is_peak, is_weekend, month, dow.

Engine	Best model	Validation AUC	Test AUC	Test F1
Spark MLlib	LogisticRegression	0.967	0.965	0.914
Python (sklearn/XGB)	XGBoost	0.984	0.983	0.853

Spark MLlib: every candidate, with the validation score it was selected on (validation AUC) beside the test score it is reported at.

Model	Hyperparameters	Validation AUC	Test AUC	Precision	Recall	F1	Accuracy
LogisticRegression	maxIter=50, regParam=0.0	0.9669	0.9648	0.9134	0.9139	0.9136	0.9139
DecisionTree	maxDepth=8, minInstancesPerNode=1	0.8125	0.8041	0.8554	0.8588	0.8451	0.8588
RandomForest	maxDepth=10, minInstancesPerNode=1, numTrees=60	0.9326	0.9314	0.8329	0.8169	0.7725	0.8169

Python: every candidate, with the validation score it was selected on (validation AUC) beside the test score it is reported at.

Model	Hyperparameters	Validation	Te	Precisi	Rec	F1	Accu
LogisticRegres	max_iter=200	0.9579	0.955	0.8044	0.79	0.79	0.907
RandomForest	max_depth=14, n_estimators=120	0.9543	0.952	0.9626	0.41	0.58	0.861
XGBoost	colsample_bytree=0.9, learning_rate=0.1,	0.9835	0.983	0.9028	0.80	0.85	0.935

Spark MLlib confusion matrix on the test quarter. Rows are what happened, columns are what the model said.

	predicted not overcrowded	predicted overcrowded
actually not overcrowded	111,097	6,300
actually overcrowded	6,869	28,643

Python confusion matrix on the test quarter. Rows are what happened, columns are what the model said.

	predicted not overcrowded	predicted overcrowded
actually not overcrowded	114,309	3,088
actually overcrowded	6,829	28,683

The Spark MLlib fails to detect 6,869 out of 35,512 truly overcrowded trips, which is a 19.3% failure rate, and reports 6,300 false positives. These two mistakes cannot be equated since the former requires extra check of the route while the latter means that there was a vehicle leaving passengers off. Such differentiation became impossible because the models showed just AUC, F1 and accuracy.

The Python model failed to detect 6,829 out of 35,512 truly overcrowded trips, which is a 19.2% failure rate, and reports 3,088 false positives. These two mistakes cannot be equated since the former requires extra check of the route while the latter means that there was a vehicle leaving passengers off. Such differentiation became impossible because the models showed just AUC, F1 and accuracy.

Dual pipeline concordance on 152,909 previously unseen trips: 94.1% with probability correlation 0.9378. Two independently trained models are not supposed to be exactly the same; high concordance means that both have learned the same thing.

Comparison	Value
Records compared	152,909
Agreement	94.1%
Probability correlation	0.9378
Disagreements	9,028
Of those, borderline	5,606
Of those, materially inconsistent	3,422
Spark correct when they disagree	2,888
Python correct when they disagree	6,140
Mean probability difference	-0.014

But the disagreements are the meat of the analysis, and those are analyzed, not tallied. The disagreements come out favoring Python (6,140 versus 2,888), which makes sense as being consistent with the gradient-boosted algorithm being superior to the neural network for this application, rather than there being any problems with either of the engines. Disagreements also have two parts: disagreements where both the probabilities are close to the cutoff point, and the models have different labels just because the cutoff has to be somewhere; and substantive disagreements, where the probabilities are well-separated, and one model has clearly gotten something fundamentally wrong. It's the latter that we're interested in, and distinguishing them is what separates a comparison from a story.

Independent training of each pipeline; exact agreement not anticipated. A high degree of agreement, along with strong correlation of probabilities, suggests learning the same underlying relationship. Every disagreement comes with an explanation in `dual_pipeline_comparison.csv`.

7.3 Delay Severity Prediction

The delay severity prediction is a classification problem having five classes within the frequency bands described in section 6.10 and is based on pre-trip features on the same chronological split. This is the most difficult problem to be solved during the project and the only problem that is not being asked by the specification but rather solved by the project.

Model	Hyperparameters	Validat	Test	Test	Test	Test
LogisticRegress	max_iter=300	0.3865	0.3017	0.3969	0.3307	0.5448
RandomForest	max_depth=16, n_estimators=120	0.4317	0.427	0.5379	0.4182	0.7339
XGBoost	learning_rate=0.1,	0.4828	0.4583	0.5128	0.4547	0.738

Three algorithms have been compared, as requested, and the choice criterion was not accuracy but the macro F1 of the validation month.

Delay is both structural and random, and the project captures both.

A regressor using the same pre-trip information has an $R^2 = 0.6312$ of delay variance explained. The residual variability is 2.08 minutes against a variability of delay of 3.69 minutes and a noise floor of 1.2 minutes for the generator. Accuracy is limited to the noise floor and not to the classifier's limitations.

The limit is measured, not stated, and this is where the meat of this section lies. Sampling the delay curve of the generator 400 times on each trip and taking the most probable class per trip achieves 80.9% accuracy. No model based solely on the pre-trip features can achieve better, since the remainder of the variation is generated after fixing the structure. The model gets 73.8%, falling 7.1 percentage points short of perfection. The problem statement requires 85% accuracy, which is above the ceiling, and 0.80 macro F1 score, which even the most accurate classifier does not achieve. Decreasing the noise of the generator would solve both problems, but it will make the dataset worse simulation.

Test month	Accuracy
10	0.7397
11	0.7414
12	0.7331

This error was seasonal and the per-month chart is the one that brought the issue to light. Training ends during the least turbulent month of the year while scoring occurs into the most turbulent one; the month was represented as a pure numeric variable and a tree can only make projections for a season it hasn't seen before, therefore the last month of the year was being predicted to behave like the least turbulent. By including the average delay for the route and for the entire network over the past 14 days in the model there was no need to project at all, since the rolling window reflects the season and not indexes it.

Per class, on the test quarter:

Class	Precision	Recall	F1
Major	0.92	0.667	0.774
Minor	0.561	0.738	0.637
Moderate	0.753	0.808	0.779
On time	0.331	0.06	0.101
Severe	0.0	0.0	0.0

The per-class table is the place where the macro F1 goal is impossible to achieve due to reasons that are completely independent of the selected algorithm. Severe scores 0 and On time scores 0.101 as a result of their rarity and presence within the range of another class, meaning that the accuracy-maximizing classifier will never generate these classes. This is not a problem that can be eliminated through tuning. Instead, this is the result of dividing a continuous noisy value into five classes with two thresholds located in the regions with the greatest density and smallest widths.

Confusion matrix for Delay. Rows are the observed class, columns are the predicted class.

	predicted	predicted	predicted	predicted	On	predicted
actual Major	15,527	26	7,710	0	0	
actual Minor	37	11,507	3,817	229	0	
actual Moderate	1,252	7,394	36,449	18	0	
actual On time	25	1,563	344	123	0	
actual Severe	40	28	96	2	0	

Features used, all pre-trip. Categorical: route_id, vehicle_type, day_type. Numeric: direction, dep_hour, planned_min, route_km, n_stops, is_peak, is_weekend, month, dow, capacity, recent_route_del, recent_network_delay.

7.4 Demand Forecasting

	MAE	RMSE	MAPE	R2
Model (HistGBR)	156.85	237.98	28.7%	0.8489
Baseline (last week)	178.25	276.33	29.28%	0.7962

Validation is a process that happens in time; the model outperforms the naive baseline on the same week day.

The naive baseline is what gives meaning to the model in this whole section. Forecasting that next Tuesday will be similar to the previous Tuesday is a very good baseline for a transit network since there is a heavy influence of weekly seasonality, and a model which does not beat it simply isn't worth using even if its absolute error looks good.

Error is reported by forecast horizon as well since one error figure can hide the rate at which the model deteriorates:

Horizon	Days	Rows	MAE	RMSE	MAPE	R2
7d	7	672	133.08	193.67	19.18%	0.8947
14d	14	1,343	147.15	212.11	27.42%	0.8815
30d	30	2,877	145.4	211.78	23.95%	0.8799

The forecast horizon is set at 14 days, which corresponds with the decision that is made, since two weeks is a general period of notice needed to shift the vehicles and staff. As error increases with the increase in horizon, as should be expected, presenting the entire curve allows the reader to see if this forecast would be appropriate for another decision.

Crowd levels are forecast using the same methodology relative to the same base, allowing for crowding forecasts to be made rather than just volume.

	MAE	RMSE	MAPE
Occupancy model	0.113	0.152	24.69%
Baseline (last week)	0.139	0.191	29.13%

7.5 Route Clustering

K-Means categorizes routes into $k=6$ operational clusters (Silhouette Score=0.4028) with good performance, overused, unreliable, and underused clusters.

cluster	n routes	avg daily demand	avg occupancy	avg delay	reliability	overcrowded trips
0	9	1285.933	0.797	5.517	0.416	2368.667
1	18	812.678	0.592	7.023	0.248	1098.833
2	17	184.035	0.209	7.008	0.264	78.765
3	19	1199.284	0.758	8.406	0.163	2162.947
4	17	423.859	0.379	4.968	0.5	514.588
5	20	560.66	0.464	9.261	0.12	777.55

Clustered on avg_daily_demand, avg_occupancy, avg_delay, reliability, overcrowded_trips, all standardised first, since the five variables are on incommensurate scales and an unstandardised K-means will form clusters almost completely based on the one with the largest range.

The silhouette of 0.4028 is the plain truth of this section and is reported rather than ignored by the neat profiles, which are produced. This means that the clusters overlap, meaning that the routes close to a border may equally well be put into either cluster; the clustering is a soft ordering of the routes, not a partitioning into discrete sub-groups. This was expected in this case, because the routes have been derived from continuous variables and there are no clear splits in the data set for a clustering to uncover. The profiles describe the network well enough, however, and the rule-based classification in 6.4 is the better justification for distinguishing two routes, which is why the clustering is reported alongside it.

7.6 Model Version Tracking

Each model, when trained, is stored with a version number, the metrics obtained from that model, the features it is built upon, and the version of libraries used. The metrics will also be written into the application database so that the model's history can be retrieved instead of recalculating it again, and past versions of the models are deleted.

7.7 Persistent Overcrowding

An overloaded route is one that operates above capacity on most days, while a route that experiences overload for a few days has incidences. A day is said to be overloaded if a certain number of the route's trips were above capacity.

Pattern	Routes
Persistent	45
Intermittent	33
None	15
Isolated	7

The two threshold values are 25% of a route's trips per day in order for a day to be considered as overloaded and 30% of a route's operating days for a pattern to be regarded as a persistent pattern with 3 days or less being categorized as isolated pattern. It is the former threshold value that was incorrect. If a day was regarded as overcrowded as long as even just a single trip in that day experienced overcrowding, most routes would be considered as persistently overcrowded and that would mean nothing.

7.8 Crowding Risk

Overcrowding Model provides a chance on a trip basis, not a class basis. 22,416 trips in the unseen time period have a chance equal to or greater than 0.75 and are hence highlighted. The risk is classified as follows.

Risk band	Trips
very low	111,175
low	9,963
moderate	9,359
high	7,423
very high	14,989

Both thresholds have been established and are published, as they are answers to different questions, and to publish one and use the other is a very subtle form of getting the answer wrong. The decision threshold is

0.5, the threshold that classification metrics are computed for in Section 7.2. The attention threshold is 0.75, deliberately higher, since a flagged-for-attention list must be small enough to go through, and if created at the decision threshold, would be too long by a factor of several. This project used to report the former and flag the latter.

Risk aggregated per route, the action-oriented view:

route id	trips	mean risk	high risk trips
R002	1739	0.3899	545
R072	1618	0.3845	542
R090	2060	0.3809	621
R069	1573	0.3727	481
R018	1583	0.3699	496
R066	2102	0.3586	651
R067	1723	0.3578	448
R025	1815	0.3557	538

8. Recommendation Engine

A total of 153 recommendations is generated by the engine with their justifications through figures. These are classified into 24 critical, 80 high, 30 medium, and 19 low recommendations. Four of these with the exact evidence of each as follows

- [Critical] Increase peak-hour frequency on R066 , occupancy 90% on average; 2,928 trips overcrowded; average daily demand 1704; 24 min of trip is above capacity; up to 11 consecutive stops are overloaded; demand +7% for the previous month; +22% for the year
- [Critical] Dwell time increase at S0141 , bottleneck rating 0.66; 7.2 min average delay on the routes served by the stop; 24% of trips are overcrowded in this stop; net accumulation 3,848 people; 64% of boardings in the peaks; serves 8 routes
- [High] Delay analysis on R063 and schedule re-timing , average delay 8.6 min; on-time performance 17%; average daily demand 1496; demand +11% for the previous month; +41% for the year
- [High] Deploy higher-capacity vehicles on R066 , occupancy 90% on average; 24 min of trip is above capacity; 2,928 overcrowded trips; deployment of a bigger vehicle means more seats and not more trips

The rules create different families of actions rather than sending out one particular message with various numbers within it, and the combination itself is a discovery about the network:

Recommended action	Count
Add trips to inside the peak window	51
Reduce frequency on and redeploy the capacity	29
Investigate recurring delays on and re-time the schedule	24
Extend dwell time	19
Smaller vehicles would serve adequately	12
Review the stop sequence	7
Increase peak-hour frequency	5
Allocate higher-capacity vehicles	5
Improve vehicle spacing to reduce bunching	1

Target	Recommendations
route	126
stop	26
network	1

Priority	Recommendations
Critical	24
High	80
Medium	30
Low	19

There were two issues that were corrected with this engine. Both these issues were producing results that appeared to be complete. Since there were only five types of actions suggested by the specification and since no rule suggested capacity change, the network that had its routes operating at 25% of their capacity was advised on frequency but never on size of the vehicles. The bottleneck recommendations came up with a consistent priority and an evidence line that was labeled as 'flagged as bottleneck' – repeating the recommendation rather than explaining it. The evidence line mentions score, delay, crowding proportion and accumulation.

8.1 What-If Scenario Impact

Each scenario is costed on more than occupancy. For R066, currently at 125% with a mean wait of 22.8 minutes, the estimated effects are:

Scenario	Occupancy	Band	Seats	Wait (min)	Coverage
Increase frequency +25%	100%	Overcrowded	624,331	18.2	1.004
Increase frequency +50%	83%	High	749,198	15.2	1.205
Add one vehicle to the roster	113%	Critical	549,412	20.7	0.883
Larger vehicles +40% capacity	89%	High	699,251	22.8	1.124
Demand grows +20%	149%	Critical	499,465	22.8	0.669
Reduce frequency -20%	156%	Critical	399,572	28.4	0.642
Remove the quietest 10% of trips	138%	Critical	449,519	25.3	0.723
Shift departures 15 min out of the peak	125%	Critical	499,465	22.8	0.803
Add a new stop on the route	130%	Critical	499,465	22.8	0.772

Impacts of operation changes, not observed measures. Demand is constant, so relief from increased capacity is an upper bound.

This is the crucial statement of the section. Demand is constant, so the relief from congestion that a particular scenario seems to provide is an upper bound: a bus line that is less crowded attracts passengers that had been skipping it, and part of the margin is used up by the change itself. The calculation is right before your eyes and not modeled, so you can verify it. 3 lines are considered, and the dashboard uses the same formula in its sliders.

By looking at two situations where the same amount of relief from crowding is provided, one will be able to understand why using occupancy is not the right way to decide between the two situations. 'Increase frequency +25%' and 'Larger vehicles +40% capacity' result in 100% and 89% occupancy, making them seem like a very close equivalent choice. However, their waiting times do not provide the same view; in the case of increased frequency, the average wait is 18.2 minutes while in the second situation, it remains 22.8 minutes, since an increase in vehicle capacity without changing the schedule is irrelevant to the person waiting at the bus stop.

9. Testing, Security & Privacy

9.1 Testing

- 208 test functions: band limits, the leak guard, model registry versioning, running and auditing logging, clean data integrity and model quality gatekeepers.
- The leakage guard stops the run if an output field gets into the feature matrix.
- Time-savvy validation keeps future data from informing past predictions.
- Role-based access determines which users can see what.
- Each step in the pipeline, metrics for models, and administrative actions get logged in the app database and form an easy to understand audit trail.
- No PII apart from anonymized IDs; unresolved taps are flagged not filtered out.

The most valuable tests assert a property rather than a value, and three are worth naming. One checks hidden-data readiness: an unseen network still classifies into more than one category, a single-route network still scores, and the headway rules flag a wide gap on an infrequent route without flagging a short headway on a frequent one. One checks that the trailing delay features only look backwards, by injecting a spike on the final day and requiring every earlier row's feature to be byte-identical. The third checks that the two modules counting bunching agree, which is the regression that had already happened once.

Quality gates two are in the suite and not in the checklist. Delay accuracy should be above the floor but at the same time, below the measured ceiling. This is because if a model exceeds the information ceiling, it means that there is a problem in the leakage. The third quality gate claims that no month should be worse predicted than the other month. This is the very same regression that was not able to detect seasonal leakage in 7.3 for such a long period of time.

9.2 Security

The passwords are encrypted using pbkdf2_sha256 hashing with 200,000 iterations, a random salt per user and constant-time string comparison. The previous version used a single round of SHA-256 with a shared salt, which is easy to compute and thus fast to break, while shared salts enable a single lookup table for all users.

Authorization uses capability-based approach. Each of the 11 views specifies what capability it needs, roles have a set of capabilities, and both web clients look at the same table. The other approach, which was implemented in the beginning of this project, had a list of tab names per each role and failed in the way that lists of names tend to do – the names diverged from the corresponding keys and four views became invisible for all the roles. Now there is a test which asserts that capabilities increase from operator to analyst to evaluator to admin, thus breaking the chain fails the test suite.

One security perimeter is stated clearly and not overstated. Static dashboard verifies the sign-in from the browser using the exported salt and hash and therefore does not pass a password through the bundle. This verification done in the client can be bypassed by anyone who reads the page. This is presentation gating. The `React` application is the one which enforces access, and the difference is mentioned both in the repository and here because confusing the two would lead to incorrect information being conveyed.

9.3 Privacy

There are no actual people in this dataset; however, it is designed as if there were any, since designing something that works for artificial data teaches one the wrong methods. The passengers are assigned opaque identifiers and there is no name, contact information, nor any payment tool associated with any passenger in the schema. There is no need to re-identify any person for the analysis presented here, since the segmentation is calculated per passenger and shown per segment and origin-destination matrix is aggregated per pairs and time periods.

9.4 Processing Log and Timing Evidence

All stage executions are logged with their times, and the run log is exported after every run. There are currently 198 recorded stage executions for a total of 5.0 hours of compute time, out of which 17 have been failures. Failures are stored, as a crash-and-recovery stage forms part of the completion process, and a success-only log would be less representative of the same thing.

Stage	Runs	Last run (s)	Best run (s)
ml-python	10	656.9	139.1
ml-delay	12	474.8	267.7
clean	10	307.5	99.2
ml-spark	9	285.8	198.4
network	16	169.5	36.0
analytics	23	125.7	15.7
ingest	8	119.0	67.4
quality	8	116.1	41.0
integrate	11	108.1	14.6
catalog	10	95.1	60.2
features	12	94.4	15.1
hdfs	13	55.1	20.1
forecast	8	7.8	3.5
compare	10	5.6	0.3
clusters	11	3.0	2.3
what-if	12	0.2	0.1
recommend	14	0.0	0.0

The two cases of poor performance were discovered through this method and not through any guesswork. Assignment of tickets was originally a per-trip Python loop and was transformed into a multinomial draw, shifting the process from minutes to seconds on 2,012,000 rows. Referential integrity checks involved collecting the keys for each parent table in a Python set and doing a membership test row by row, which was converted to a broadcast anti-join. These are both the same error in disguise.

10. Limitations & Future Enhancements

10.1 Limitations

- Delay forecasting is constrained by the residual dispersion mentioned in 7.3, and not the choice of classifier.
- The data set is synthetic, and actual feeds will introduce more complications.
- All the figures presented are for individual nodes, not clusters.
- Bottleneck identification is impossible to validate using ground truth from this data set due to the explanation in 6.5.
- There is overlapping among route clusters, meaning that cluster identity is just an informative description.
- “What If” analysis assumes constant demand and thus exaggerates improvement potential.
- Further research: Live data acquisition, geospatially sophisticated routing, passenger-customized recommendations, and automatic scheduling.

The fundamental constraint is the one which is most difficult to ignore: the data set is made up, and thus the analyses carried out here are fundamentally tested against a world which the project itself wrote the rules for. It cuts both ways – it is what makes validation even possible in the first place, since the events and the labeled stops provide the scoring against which the detectors are tested, but it also makes the achievement of any kind of score not as impressive as it could seem, because the system to be recovered has been engineered to be recovered. The solution taken here is to draw attention to the failures just as much as the successes.

The second limitation is that some results appear to be cause and effect relations when actually they may be merely descriptive. Bottleneck scores indicate that a station is related to delays, congestion, and crowding, but that is all. They do not indicate that the station causes these things, and even a perfectly good station located in a corridor that is suffering from congestion will receive a high bottleneck score. The suggestion that dwell times should be extended is based on evidence, which has been presented along with the suggestion, so that an experienced corridor operator can reject it.

10.2 Future Enhancements

The first extension I would recommend adding is streaming ingestion, as it alters the nature of the solution rather than the quality of it. All the findings here are retrospective: using the crowding model with the departure-time feed will alert on a journey before it departs, meaning that there is still time for the operator

to act. The list of features here is already limited to those that describe the journey prior to departure, so the algorithm itself wouldn't need any adjustments, only the data source.

The second extension is real geography. Stops have coordinates and routes have distances, yet there is no map of roads, so none of these systems can tell anything about a detour, an alternative route, or even a shared corridor between two routes. Bottleneck detection could benefit greatly from it: a corridor-based system would allow to distinguish the stop that is causing congestion from the downstream stop.

11 Conclusion

UrbanTransit IQ's challenge was that of a tougher question than one that can be answered by using a spreadsheet: it was not only about what route had the highest number of passengers, but rather about what routes were hidden in trouble – overfilled at certain hours, underfilled at other times or simply delayed for unknown reasons. Over the entire simulated year of operation, the software managed to identify such trends and generate concrete suggestions.

The project satisfies all the functional and non-functional requirements specified in the SRS. There are two different pipelines – one using Apache Spark/PySpark/Spark MLlib and another one using Python and scikit-learn/XGBoost which independently solve the same overcrowding prediction problem and give the same answer about the majority of unseen trips, which is the real proof of correctness of the analysis carried out by the project, not just the result of one and unvalidated model. Demand forecasting outperforms the baseline, routes are grouped into clusters by their operating characteristics, and the recommendation engine provides recommendations backed up with evidence.

In addition to the strengths mentioned, the project also acknowledges its weaknesses openly. The delay-severity analysis fails to achieve the 85% accuracy benchmark set by the SRS. Rather than tampering with the simulated dataset to artificially increase this percentage, the project team has found the actual maximum possible value of accuracy mathematically considering the stochasticity of delay. It is fair to say that such an approach is even more proof of technical rigor than achieving a perfect number through suspicious coincidences.

Thus, UrbanTransit IQ shows how a pipeline in Big Data and Data Science can process a whole year worth of complex transportation data and produce actionable information for the transport authority within just one month.